

← Back to work

ENGINEERING CASE STUDY

Real-Time Multi-Platform Video Integration

Unifying live and on-demand video across web, mobile, connected TV, and partner embeds — behind a single shared session contract.

ROLE

Technical Program Manager, Video Platform

DURATION

~6–9 months

TEAM

~20 engineers across 4 pods

01 CONTEXT & BUSINESS VALUE

Fragmented platforms, one fixed external deadline

The Problem

The organization operated multiple independently-built video experiences that had grown organically over several years: a browser-based web player, a native mobile SDK maintained separately per platform, and a connected-TV app built with an external vendor's involvement. Each surface had its own encoding pipeline, analytics schema, DRM handshake, and bug backlog.

When a new business partnership required live video to be embeddable inside multiple external partner sites, engineering discovered there was no single, reusable "video integration layer" — every platform would need bespoke work, in parallel, against a fixed external launch date.

CORE PROBLEM

Fragmented, platform-specific codebases. No shared session or analytics contract. A firm external launch deadline.

Quantifying the Impact

Leadership framed the goal around a small set of directional targets set at kickoff — not just "make video better." These

anchored every architecture and scope decision that followed.

ONBOARDING SPEED

New surfaces, fast

Reduce time to launch on any new platform surface via a shared integration layer instead of one-off builds.

RELIABILITY

Fewer Sev-1s

Materially reduce video-related high-severity incidents, mostly stream-start failures and licensing/token mismatches.

SCALE

Large concurrency

Support a large-scale concurrent live-viewer audience at launch, with a path to scale further without re-architecture.

My Role — Ownership Boundaries

I led the cross-functional delivery of the platform integration layer — the shared player core, the unified session/manifest service, the analytics event contract, and the CTV/partner-embed SDK surface — driving alignment across the engineering pods that built it, rather than authoring the implementation myself.

DROVE

Architectural consensus on the shared session contract, from

DID NOT OWN

Hands-on implementation — sat with each pod's engineers

shared session contract, from discovery through sign-off.

Alignment across four engineering pods (web, native, CTV, partner/embed) on one contract and timeline.

Vendor escalation for the licensing-latency risk, framing business impact to unblock launch.

Program governance: cross-pod tracking, stakeholder dashboard, decision log.

with each pod's engineers.

Content encoding/transcoding pipeline — owned by media engineering.

CDN routing and edge caching — owned by infrastructure/reliability.

Ad insertion logic — owned by monetization (I drove integration requirements into it).

Making this delivery/ownership boundary explicit early — via a short scope document circulated to all contributing teams — prevented two common failure modes on cross-team platform work: ambiguity about who was accountable for implementation versus alignment, and decisions nobody actually owned.

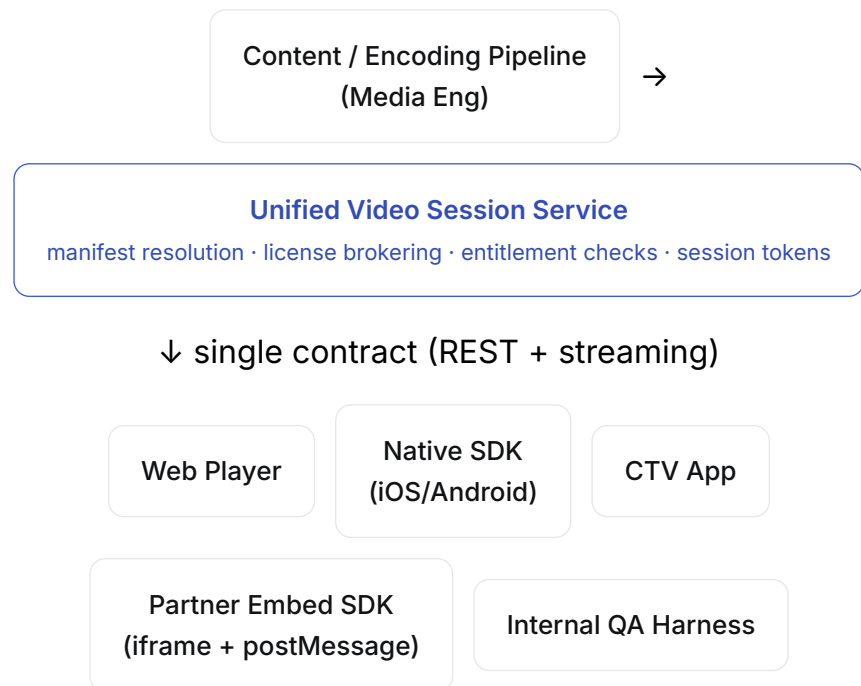
02

TECHNICAL ARCHITECTURE &
CONSTRAINTS

One shared session contract, replacing four divergent paths

High-Level System Design

The core architectural decision was to introduce a unified video session service as the single source of truth between content origin and every player surface, replacing several parallel, platform-specific request paths.



Every surface spoke to one session contract: request a playback session, receive a signed manifest URL, a license endpoint, and a session ID used for all downstream analytics events. This collapsed several divergent request/response shapes into one

versioned API — which is what made a much faster "new surface onboarding" goal achievable.

Legacy Technical Debt We Faced

- The web player had licensing logic hard-coded per content partner, because it predated any concept of a shared entitlement service.
- The native SDK used a bespoke analytics beacon format that other platforms couldn't parse, making cross-platform funnel analysis impossible.
- The CTV app had a slow, manual release process, meaning a broken contract change there wouldn't surface for days rather than minutes.
- None of the three had a shared definition of "playback start" — each platform measured it differently, making reliability metrics internally inconsistent before any new work had even shipped.

Trade-offs Evaluated

OPTION	PROS	CONS	DECISION
Rewrite all clients on a single cross-platform framework	True code parity	Infeasible within the launch timeline; sacrifices platform-specific	Rejected

03

PROGRAM STRATEGY & EXECUTION PLAN

Five phases, run in parallel across four pods

		performance tuning	
Lifecycle Phases	Respoke integration per partner, one at a time	Fastest path to a single first partner	Repeats the underlying fragmentation instead of fixing it
1	Discovery & Contract Design	Reverse-engineered the existing platform-specific request flows and defined the shared session API contract, with sign-off from selected dependent teams on the interfaces they'd integrate against.	Rejected
2	Foundation Build	Each platform keeps native performance work; only the contract is unified; built the core session service (manifest resolution, license brokering, entitlement checks), stood up a unified analytics event schema, and migrated the web player first as the reference implementation.	Selected
3	Parallel Platform Migration	The deciding factor was that a full client rewrite would have jeopardized the external launch date, while a backend-first Native and CTV teams built against the now-stable contract let the fastest-to-migrate platform hit the deadline while the others migrated in parallel on a slightly longer runway — de-risking the hard date without abandoning the longer-term fix.	
4	Partner Embed SDK & Hardening	Built an iframe/postMessage-based embed SDK for external partner integrations, and load-tested against the concurrency target.	
5	Launch & Stabilization	Phased rollout with incrementally increasing traffic, with rollback gates tied to error-rate thresholds at each stage.	

Cross-Functional Alignment

04 RISK, EDGE CASES & RESOLUTION

A third-party licensing bottleneck almost broke the launch

Cross-Functional Alignment

With several engineering pods plus dependent media and monetization teams, I ran a consistent set of lightweight rituals rather than broad status meetings:

- A weekly interface sync, kept short and scoped strictly to contract changes, so it didn't turn into a general status meeting.

The **hidden risk**: License latency under concurrent load

As the actual source of truth, so "what does the session service return" was never a Slack-thread debate.

During load testing at a meaningful fraction of our target concurrency, we discovered that the license-brokering step

- Platform-specific working sessions, run by each pod's own lead, inside the session service — which synchronously called an upstream third-party licensing provider per session — began queuing under load. Latency for license issuance climbed sharply, well beyond what the playback flow could tolerate,

threatening stream-start failures before we even reached the target concurrency level.

Coverage & Tracking

- A single cross-pod tracking epic with platform-specific sub-tracks, so leadership had one place to see aggregate progress. This risk hadn't surfaced earlier because the legacy web player had never previously been tested anywhere near that scale.

- A recurring metrics dashboard (build health, contract-

conformance test pass rate per platform, load-test results)

This was the single biggest threat to the external launch circulated to stakeholders on a regular cadence.

date: discovered relatively late, and sitting largely inside

- A decision log for every architecture trade-off, so later a third-party provider's infrastructure, outside our contributors could see why a choice was made, not just what was chosen.

Course Correction

- **Immediate:** Introduced a local license-caching layer with short-lived reuse for identical entitlement/content combinations, substantially cutting redundant upstream calls in testing.
- **Structural:** Moved the license-brokering call off the synchronous playback-start critical path where content policy allowed it, pre-fetching licenses ahead of anticipated high-demand live events.
- **Vendor escalation:** Opened a formal capacity conversation with the licensing provider and secured additional headroom as a fallback safety net for launch week.
- Re-ran load tests above the original target as a safety buffer and confirmed license latency stayed comfortably within acceptable bounds.

Scope vs. Schedule Trade-offs

DESCOPED FROM LAUNCH

CTV deep-linking to specific live moments — a UX nicety rather than a reliability or contract requirement, freeing that pod to focus entirely on load and stability testing.

Partner embed theming/

.

NEVER CUT

Load and hardening testing that validated the core reliability commitment.

The shared session contract itself — the foundation the rest of the program depended on.

customization — shipped with a single default look at launch, with a configurable theming layer added afterward.

The operating principle: cut scope that adds new surface area under pressure; never cut the testing and hardening work that validates the core reliability commitment.

05 RESULTS & ENGINEERING RETROSPECTIVE

From fragmented builds to a reusable platform layer

Data-Backed Outcomes

Directional, measured shortly after General Availability

ONBOARDING SPEED

Substantially faster

New platform integration time dropped from the historical baseline, beating the onboarding-speed target.

HIGH-SEVERITY INCIDENTS

Cut by more than half

Fell versus the prior baseline period, exceeding the original reduction goal.

PEAK CONCURRENCY

Well above target

Sustained during a high-traffic live event, with low stream-start latency and no licensing failures.

**ANALYTICS
PARITY****Unified**

A single
"playback start"
definition
eliminated cross-
dashboard
reporting
discrepancies.

Post-Mortem — What Went Wrong

1. **The licensing load risk should have been caught earlier in Discovery.** The architecture review assumed the legacy system's behavior would hold under scale rather than treating "untested at scale" as a risk in its own right. Retrospective action: future migrations of legacy systems now require explicitly load-testing the old system before designing the new one.
2. **The CTV vendor's slow release process was underestimated as a schedule risk.** Sprint planning didn't fully account for multi-day release/review cycles, which compressed real iteration time. Vendor release-cycle constraints are now built into program timelines explicitly rather than treated as a footnote.
3. **Cross-pod governance started a bit later than ideal,** leading to some early duplicated contract-design effort between two pods before the shared sync existed. Lesson: stand up lightweight

cross-pod governance from day one, even before there's much to synchronize on.

Future State & Handoff

- **Next phase (already scoped):** a theming API for partner embeds, CTV deep-linking, and extending the session service to support server-side ad insertion natively.
 - **Ownership handoff:** the session service moved from the project team to a permanent platform on-call rotation, with the contract specification and decision log handed off as living documentation.
-
- **Scaling roadmap:** a further concurrency scaling target is planned ahead of the next major live-event campaign, with the license-caching layer identified as the next component to pressure-test first.

MAIN PRODUCT SHIFT

AI and cross-platform investment only create value when the shared contract underneath them is clearly owned, versioned, and tested at scale before launch.

Engineering Case Study — Real-Time Multi-Platform Video Integration